Thread Performance Across Different Languages: C vs Rust

A Study on Benchmarks of Parallel Workloads

Erik Vicente Flores-Siemsen

CSCI 440 Operating Systems

California State University, Chico

Spring 2026

1. Approved Question

How does the performance of threads in C compare to Rust across CPU intensive tasks?

To answer this question, I took the three preapproved parallel algorithms(Monte Carlo, Matrix Multiplication, DNS) and implemented them all in both C and Rust. In C i utilized pthreads, and Rust i used std::thread.

I then benchmarked them across multiple data sizes:

Monte Carlo Pi Estimation: Each thread independently samples random(x, y) points to estimate Pi.
I tested with 1,000,000 , 10,000,000 , and 100,000,000 points.

Matrix Multiplication: Two square N x N matrices are multiplied together, each thread in charge of a single row.
I tested with 256 x 256 , 512 x 512 , 1024 x 1024

DNS Resolution: Requester threads push hostnames into a bounded queue, then resolver threads take those hostnames and call getaddrinfo(). The resulting list is written to an outfile of hostnames and ips.
I tested with 50, 200, and 500 hostnames respectively.

All tests used 4 threads to match the physical cores on the machine. The number of threads was the constant while the data size varied.

2. Experimental Design

2.1) Hardware and Operating System

Type of Component	Test Machine
Cpu	Intel Core i7-8569U @ 2.80GHz
Physical Cores	4
Logical Cores	8(hyper threading)
RAM	16gb
Operating System	macOS 15.7.5 (Sequoia, Build 24G624)
Compiler (C)	Apple Clang via gcc, -O2 optimization
Compiler (Rust)	rustc via cargo build --release

2.2) Language and Thread Models

C uses POSIX threads(pthreads) via pthread_create() and pthread_join(). The shared state is protected with a mutex. The standard rand() function is used for random number generation. For Monte Carlo's massive points needed, I seeded per-thread with srand(time(NULL) ^ (tid * 1000003)). 1000003 being a prime number.
Rust used std::thread via std::thread::spawn() and JoinHandle::join(). The shared state is also protected with a Mutex.

No external crates were used in Monte Carlo or Matrix Multiplication, only Rust's standard library. The DNS rust implementation uses the dns-lookup crate to wrap getaddrinfo().

2.3) Compilation

Equivalent optimization levels were used for both languages to get as close to a fair comparison as possible:

C: gcc -O2 -o <binary> <source>.c -lpthread -lm

Rust: cargo build --release

Using -02 for C and --release for Rust represents standard optimization levels for both languages.

2.4) Potential Noise Factors

Thermal Throttling: My i7-8569U is a mobile processor. Under a sustained load, CPU frequency may drop as the chip heats up. This has been a frequent problem for me and my computer usually sounds like an airplane is taking off. To reduce this offset I ran 50 trials per test case to average over this effect and maintain an acceptable margin of error.

DNS caching: macOS catches DNS responses via mDNSResponder. Repeatedly resolution of the same 50 hostnames within a short window could return cached results, reducing network latency. The effect on DNS benchmarks is discussed in the results.

Background Processes: Tests were run with no other significant processes running. Browser, other applications, any minimized windows were all fully closed during data collection

Thread count and Physical Cores: I learned that although my computer claims it has 8 cores those are only logical cores, and using all 8 would introduce contention on shared execution units, possibly mess with my performance comparisons. For this reason I decided to use only my 4 physical cores by matching thread count to 4.

3. Testing Procedure

3.1) Timing Method

Execution time was measured using

C: clock_gettime(CLOCK_MONOTONIC, &start) / clock_gettime(CLOCK_MONOTONIC, &end)

Rust: std::time::Instant::now() / start.elapsed().as_secs_f64()

CLOCK_MONOTONIC was chosen because its unaffected by the system clock adjustments, it measures real elapsed time rather than CPU time, in hopes to directly capture parallelism benefits since multiple threads contribute to wall-clock progress simultaneously.

3.2) Benchmark Script

All benchmarks were run using a shell script(scripts/run_all_tests.sh) it executes each test case(the different data sizes) 50 times and extracts the elapsed time from stdout into a CSV file. Each CSV contains the columns: Trial, Time_seconds.

Data analysis was performed by a C program(scripts/analyze.c) that reads the CSVs computes statistics and generates SVG bar charts. I know it couldve been easier doing it another language, such as python, but Im not too good at python and I wanted all tools used in the project to remain consistent.

3.3) Tests Visualized

Algorithm	Language	Data Sizes	Trials
Monte Carlo Pi	C + Rust	1M, 10M, 100M	50 each
Matrix Multiply	C + Rust	256, 512, 1024	50 each
DNS Resolution	C + Rust	50, 200, 500	50 each

Total: 18 test cases x 50 trials = 900 individual benchmark runs

4. Test Results

All results stem from my n = 50 trials. Margin of error z score was 1.96 (95% confidence interval). Time is in seconds (s)

4.1) Monte Carlo

Test Cases	Mean(s)	Std dev	MoE	Moe%	Min(s)	Max(s)
C / 1M	0.009282	0.000893	0.000248	2.7%	0.007381	0.011128
Rust / 1M	0.000760	0.000060	0.000017	2.2%	0.000675	0.000888
C / 10M	0.088859	0.009827	0.002724	3.1%	0.074977	0.101438
Rust /10M	0.005854	0.000351	0.000097	1.7%	0.005515	0.006852
C / 100M	0.910658	0.100991	0.027993	3.1%	0.748570	1.025423
Rust / 100M	0.061847	0.002453	0.000680	1.1%	0.059364	0.072373

Rust outperforms C by 12-15x across all data sizes. All margins of error were under 3.1%.

4.2) Matrix Multiplication

Test Cases	Mean(s)	Std dev	MoE	Moe%	Min(s)	Max(s)
C / 256x256	0.006938	0.000722	0.000200	2.9%	0.006126	0.008787
Rust / 256x256	0.009224	0.001242	0.000344	3.7%	0.008086	0.015644

C / 512 x 512	0.057222	0.003641	0.001009	1.8%	0.053852	0.073797
Rust / 512 x 512	0.072661	0.005330	0.001477	2.0%	0.067646	0.094394
C / 1024 x 1024	0.539160	0.031375	0.008697	1.6%	0.494122	0.606460
Rust / 1024 x 1024	0.654402	0.094323	0.026145	4.0%	0.570169	0.914540

C outperformed Rust by about 21-25% across all matrix sizes. All margins of error are under 4%, acceptable results

4.3) DNS Resolution

Test Cases	Mean(s)	Std dev	MoE	Moe%	Min(s)	Max(s)
C / 50 hosts	0.036419	0.103351	0.028647	78.7%	0.020251	0.752495
Rust / 50 hosts	0.021057	0.000676	0.000187	0.9%	0.019970	0.024162
C / 200 hosts	0.088811	0.046314	0.012838	14.5%	0.072692	0.349812
Rust / 200 hosts	0.077245	0.003978	0.001103	1.4%	0.072542	0.097934
C / 500 hosts	0.187866	0.010761	0.002983	1.6%	0.176497	0.239272

Rust / 500 hosts	0.287667	0.583817	0.161826	56.3%	0.175328	4.328346

My DNS results went a little haywire. C at 50 hosts had a whopping 78.7% MoE, and Rust at 500 hosts spiked to 56% MoE. These spikes are attributed to massive DNS timeouts that threw the rest of the trials off. I did rerun the code but decided to include these to show the limitations and possible interference of network on these tests. The 78.&% is more indepthly analyzed in my Analysis section.

4.4) Accuracy and Confidence

All non-DNS tests cases remained within the appropriate margin of error, ranging from 1.1% - 4.0%, my target range be 5% - 10%. The z-score corresponds to a 95% confidence interval meaning we are about 95% confident that the true mean execution time falls within acceptable limits.

Doing 50 trials per test case is a sufficient sample size to achieve stable means. The standard deviations of the CPU bound workloads were consistently small relative to the mean, so that confirms there was low noise in the measurements and my precautions were effective.

5. Analysis

5.1) Monte Carlo: Rust is ~14x Faster

The magnitude of Rust's advantage over c in Monte carlo(12~15) can be attributed to the difference in random number generators, not fundamental language performance. The c version I utilized rand(), which in glibc and Apples libc maintains a global seed protected by a lock. Even though each thread called srand() separately, the underlying rand() implementation serialized access to shared state, which creates contention between threads

The Rust version of the code uses a custom (Linear Congruential Generator) implemented as a local function with no global state. Because of that each thread maintains its own seed in a local variable and advances it. There is no lock but there is no shared state or contention, the RNG cost is a minute amount of arithmetic operations per call

Rust and its ownership model encourages thread local state functions, while in C its easier to use the global rand() although it could be a trap. If i rewrote this code I would use a local rand like rand_r() maybe to narrow the difference in performance.

MoE was under 3.1% across the board.

5.2) Matrix Multiplication: Why C is ~23% Faster

In the test trials the C code was consistently ahead of rust in Matrix Multiplication(21~25%), this I attribute to compiler optimization. I used a triple nested loop (i, j, k) which i found is a well studied pattern that GCC aggressively vectorizes using SIMD instructions(SSE/AVX). With -O2, the C compiler applies auto-vectorization to the innermost loop, processing multiple floating-point multiplications per clock cycle.

Rust's release mode applies similar optimization, but taking into account the presence of a small unsafe block(i passed non overlapping row pointers across thread boundaries) it coulve inhibited some vectorization passes. Since all my rust code was based off my C code I was more confident in, I used raw pointer arithmetic equivalent to my C approach.

The flat 1D array layout (row-major, A[i*N+k]) used in both implementations ensures cache-friendly access patterns. Both versions allocate contiguous memory and stride through rows linearly, which is optimal for CPU cache line prefetching.

MoE was under 4% and confidence intervals do not overlap between Rust and C on any matrix size. Statistical Significance.

5.3) DNS Resolution: Network Variance and Caching

DNS is fundamentally different from the other two algorithms, Monte Carlo and Matrix Mult. are inherently CPU-bound and involve heavy computation time. DNS, in both C and Rust, spends a significant amount of time waiting for Network responses from external DNS servers. It makes it hard to manage and benchmark language level performance differences.

Looking at the tests you can see at 50 hosts the C code completely spikes and had extremely high variance. At 200 hosts it still had a massive 14.5% margin of error but that is nowhere near the ridiculous 78.7% originally I thought I had just an unlikely DNS Timeout spike, but when I reran all 50 tests I got a similar high 70s% MoE. I considered leaving it out of the data as its not representative of what the code/language is capable of but I included it because it was still real data, and happened in my safe environment. I concluded that at smaller host counts, the C implementation spent a disproportionate amount of time in the random sleep polling loop waiting for queue space, rather than doing meaningful DNS work. Its a limitation of the spin-wait approach compared to the Condvar-based blocking used in the Rust version.

An important note on DNS benchmarking on a macOS system is that the DNS cache (mDNSResponder) is holding onto the recently fetched names and would therefore speed up and muddle my results between tests. The benchmark uses a fixed list of 50 hostnames cycled repeatedly, by the second or third time through the list, many responses are now cached locally and return in microseconds rather than round trip time. This explains the counter intuitive result where in the 200-host C trial(cycles list 4 times) was similar in time to the 500-host trial(which cycles it 10x). The cache starts to absorb most of the work.

If i were to take only the 4 clean DNS cases, (rust/50, Rust/200, c/500, c/200) and show them at scale, we would see that both languages perform similarly. An expected result for a network bound workload.

5.4) Summary of Statistical Significance

Algoritm	Winner	Speedup	Significant?
Monte Carlo	Rust	12-15x	Yes, MoE ranges acceptable
Matrix Mult	C	21-25x	Yes, MoE ranges acceptable
DNS(200, 500 hosts)	Tied	<15%	No, too much noise within network variance to be sure

6. Conclusion

The original question: How does thread performance in C compare to Rust?
The answer: There is no single right answer.
The Results show that performance depends heavily on the workload rather than the language.

As you can see in the table above

- Monte Carlo
    - Rust is Dramatically Faster ~14%
    - This is due to Rusts natural encouragement of thread-local state and avoids the contention C rand into
- Maxtrix Multiplication
    - C is faster by ~23%
    - GCC vectorization of the triple nested loop outperforms Rust's more conservative treatment of unsafe pointers acorss thread boundaries
- DNS
    - Both languages performed comparably.
    - The bottleneck was of course network latency, the threading model differences had no statistical significance

These findings are consistent with the broader system programming models. Rust's zero-cost abstraction and ownership system provide performance equal to or exceeding C in cases where C code relies on a global mutable state. C's compiler toolchain ca still outperform Rust in tight computational loops where unsafe code limits optimization.

Both languages successfully parallelized all three workloads, and both demonstrated clear speedup over what a single threaded implementation would achieve. The key insight is that language choice matters most when the workloads performance depends on how shared state is managed.

7. Learning Outcomes

This project as very insightful was well as fun, I enjoyed learning and researching into these algorithms and how many different ways there are to do them.

Here are several insights I found within the benchmarks.

Rusts ownership model is not only a safety feature, it is a performance shaping architecture.
Having to create a thread-local LCG in Rust as a natural result of following ownership rules actually forces you to optimize. C's global rand() contention is a natural result of following the path of least resistance. This experience helped me make the connection between language design and runtime behavior. To remain intuitive.

Statistical Rigor and Environment Shaping matters enormously for benchmarking
The DNS results and the trouble I had with caches to try and explain them made me do a real deep analysis of my testing environment, how many tests would be feasible to get a confident answer, and the outside influences that affect all programs. Running 50 trials and computing standard deviation makes any outliers as clear as day, and incorporating them into the conclusion not on accident, but as honesty about the nature of the program.

Workload Characterization should precede implementation
Choosing CPU-bound, memory-bound, and i/o bound algorithms produced a much more informative comparison than three extremely similar ones. I think the DNS results being tied and the Matrix multiplication being up to 25% faster are both equally informative pieces of information

8. References

[1] Dhakal, S. (2024). Estimating Pi: A Monte Carlo Approach Enhanced by OpenMP Parallelism. Medium. https://medium.com/@suyash.dhakal2/estimating-pi-a-monte-carlo-Approach-enhanced-by-openmp-parallelism-25d3117f6ceb

[2] ishanthilina. (n.d.). Pthreads-Monte-Carlo-Pi-Calculation [GitHub Repository]. https://github.com/ishanthilina/Pthreads-Monte-Carlo-Pi-Calculation

[3] Pfosi, J. (n.d.). A Comparison of Concurrency in Rust and C. Tufts University. https://ehnree.github.io/documents/papers/rustvsc.pdf

[4] Davis, E. (2025). High Performance Matrix Multiplication. arXiv. https://arxiv.org/pdf/2509.04594

[5] imsure. (n.d.). parallel-programming/matrix-multiplication [GitHub Repository]. https://github.com/imsure/parallel-programming

[6] Albaqeri, A. (n.d.). Parallelizing-Matrix-Multiplication [GitHub Repository]. https://github.com/Anas-Albaqeri/Parallelizing-Matrix-Multiplication

[7] Cosmo, R., & Zacchiroli, S. (2022). Comparing C and Rust on Multicore Architectures. arXiv:2107.11912. https://arxiv.org/abs/2107.11912

[8] Xu, Z. et al. (2022). Benchmarking Rust Performance Against C++. arXiv:2209.09127. https://arxiv.org/abs/2209.09127

[9] zk7. (n.d.). parallel-dns [GitHub Repository]. https://github.com/zk7/parallel-dns

[10] Wailes, C., Chen, W., & Sayler, A. (2012). CSCI 3753 Programming Assignment 2 — queue.c, queue.h, util.c, util.h. University of Colorado Boulder.

[11] Rust Documentation. (n.d.). Fearless Concurrency. The Rust Programming Language Book. https://doc.rust-lang.org/book/ch16-00-concurrency.html

[12] Rust Documentation. (n.d.). std::sync::Condvar. https://doc.rust-lang.org/std/sync/struct.Condvar.html

[13] Biriukov, V. (2025). getaddrinfo() from glibc. https://biriukov.dev/docs/resolver-dual-stack-application/4-getaddrinfo-from-glibc/

[14] Flores-Siemsen, E. (2025). multi-lookup.c / main.rs — Multi-threaded DNS resolver. CSCI 440, California State University, Chico. Prior course assignment adapted for this project. Source code included in dns/c/ and dns/rust/ directories of submission.